

# Predictive Maintenance System

Remaining Useful Life (RUL) Estimation for Inverter / UPS Components

Relays · Contactors · MOSFETs · LCD · Switches

Technical Solution Document | March 2026

## Executive Summary

This document presents a structured approach to estimating and predicting the remaining useful life of critical components in inverters and UPS systems — primarily relays and contactors, extended to MOSFETs, LCD backlights, and mechanical switches. The system operates at two levels: an edge layer on the inverter for real-time local prediction with LED/buzzer alarms, and a cloud/server layer for fleet-wide ML-driven forecasting via MODBUS and MQTT protocols.

| 2-Layer Architecture | 5 Components | MODBUS + MQTT | Edge + Cloud ML |
|----------------------|--------------|---------------|-----------------|
| Edge + Cloud         | Monitored    | Protocols     | Intelligence    |

# 1. Problem Statement

Inverters and UPS systems rely on electromechanical and electronic components that degrade progressively over time. When these components fail without warning, the result is unplanned downtime — which in industrial and critical-power environments can be extremely costly. Standard inverter designs provide no visibility into the health of individual components.

- No component-level health monitoring in standard inverter firmware
- Failures are detected only after they occur — not before
- Maintenance is either time-based (wasteful) or purely reactive (costly)
- No mechanism to collect degradation data for fleet-wide learning
- Field technicians lack early warning to plan preventive intervention

## Objective

Goal: monitor the health of each component in real time, estimate its remaining useful life continuously, raise pre-alarms before failure, and send data to the cloud for ML-driven prediction across the entire fleet.

# 2. Core Concept — Remaining Useful Life (RUL)

RUL is the central metric of this system. It represents how much operational life a component has left before expected failure. It can be expressed as a health index percentage (100% = new, 0% = failed), a remaining cycle count, or estimated hours/days to failure.

Three fundamental questions drive the design for every component:

|                                  |                                                                                                    |
|----------------------------------|----------------------------------------------------------------------------------------------------|
| What kills it?                   | Identify the dominant failure mechanism: arc erosion, thermal fatigue, lumen decay, etc.           |
| What can you measure?            | Find a physical variable correlating with damage accumulation: current, temperature, cycles.       |
| What does end-of-life look like? | Define the measurable failure marker: rising contact resistance, lumen below 70%, threshold drift. |

# 3. System Architecture

The solution operates at two independent layers. The edge layer provides real-time local intelligence and must function even without network connectivity. The cloud layer provides fleet-wide ML forecasting and maintenance automation. Both compute RUL independently; the cloud refines edge thresholds over time via OTA updates.

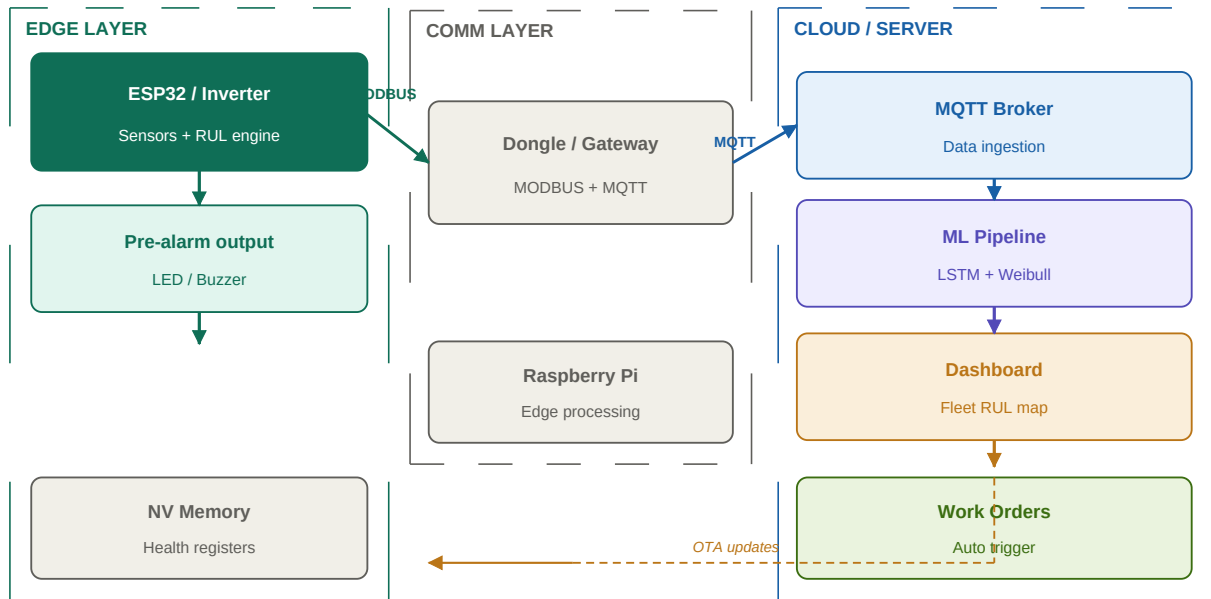


Figure 1: Two-layer system architecture — Edge (ESP32/Pi) and Cloud/Server with OTA feedback

### 3.1 Edge Layer (ESP32 / Inverter MCU)

- Continuously counts operating cycles and measures stress parameters for each component
- Runs lightweight degradation models to compute a health index per component
- Maintains health registers in non-volatile memory — data survives power cycling
- Drives physical alarm outputs: green/amber/red LEDs and buzzer with no cloud dependency
- Exposes all health data as MODBUS registers for the gateway to read on demand

### 3.2 Gateway Layer (Raspberry Pi)

- Polls inverter MODBUS registers on a configurable interval (5 to 60 seconds)
- Runs more capable RUL models than the edge MCU can handle
- Publishes timestamped JSON payloads to the MQTT broker
- Buffers data locally when cloud connectivity is unavailable — no readings lost

### 3.3 Cloud / Server Layer

- Ingests data from all fleet devices into a time-series database
- Runs LSTM anomaly detection and Weibull survival analysis
- Serves a live fleet dashboard for operations and maintenance teams
- Automatically triggers work orders when RUL drops below a configurable threshold
- Pushes refined degradation thresholds back to edge devices via OTA

## 4. Component-Specific Degradation Models

Each component type has a distinct primary failure mechanism and therefore requires its own degradation model. The table below summarises all monitored components.

| Component         | Failure Mode                     | Key Variables                      | Algorithm                      |
|-------------------|----------------------------------|------------------------------------|--------------------------------|
| Relay / Contactor | Contact erosion, coil insulation | I_switch, N_ops, T_coil, R_contact | Wear accumulation + Arrhenius  |
| MOSFET            | Bond-wire thermal fatigue        | Delta_Tj, T_jmax, I_drain          | Norris-Landzberg power cycling |
| LCD Backlight     | LED lumen depreciation           | On-hours, T_ambient                | L70 lumen curve + temp factor  |
| Switch (mech.)    | Contact friction and arcing      | N_actuations, bounce time          | Cycle count + bounce monitor   |
| Electrolytic Cap. | ESR rise, capacitance drift      | Ripple current, T_ambient          | Arrhenius capacitor model      |

Table 1: Component failure modes, measured variables, and algorithms

### 4.1 Relay and Contactor Model

Relays are the most failure-prone components in inverter systems. Two independent aging mechanisms must both be tracked and the worse of the two determines the overall health index.

#### Contact wear (primary mechanism):

Every switching event creates an arc that erodes the contact surface. Arc energy scales approximately as  $I^2 \times V$ . A damage accumulator tracks total arc energy over the relay's lifetime. When accumulated damage reaches the rated capacity from the component datasheet, the contact is end-of-life.

- Damage per operation: proportional to  $I^2 \times V$  at the moment of switching
- Stress factor  $K(I,V,T)$ : scales damage when switching occurs above rated conditions
- Health index: 1 minus (accumulated damage divided by rated damage capacity)

#### Coil insulation aging (secondary mechanism):

The coil insulation degrades via thermal aging following the Arrhenius equation. For every 10 degrees C rise above reference temperature, insulation lifetime roughly halves. The coil age factor is accumulated on each operation, weighted by the Arrhenius rate at measured temperature.

- Arrhenius rate =  $\exp((E_a / k) \times (1/T_{ref} - 1/T_{measured}))$
- Activation energy  $E_a$  approximately 0.7 eV for typical coil insulation
- Final relay HI =  $\min(HI_{contact}, HI_{coil})$  — worst case always wins

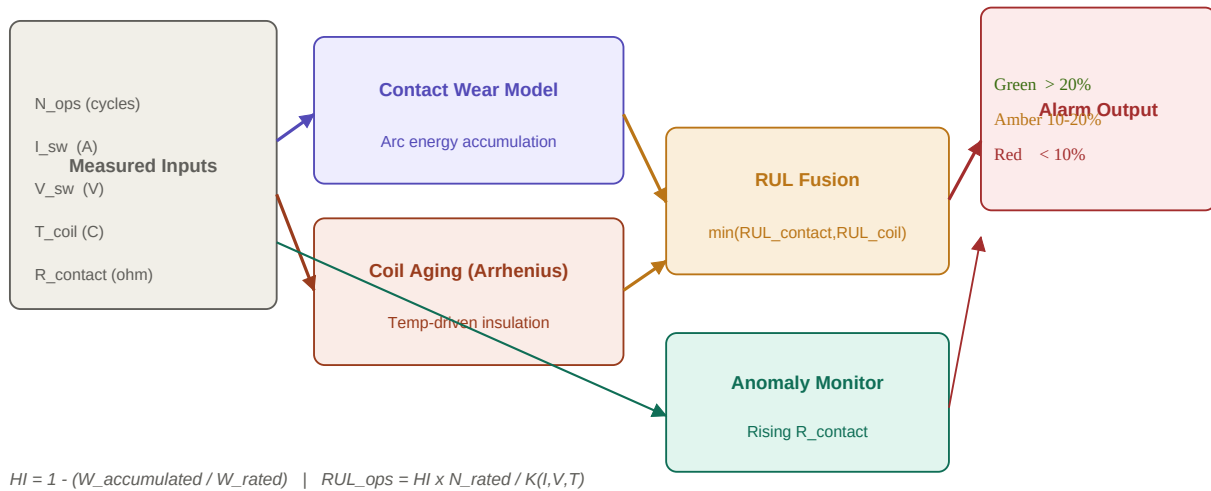


Figure 2: Relay RUL algorithm — dual-path wear model with anomaly override and alarm output

### Anomaly override:

Contact resistance rising to three times its baseline value triggers a critical alarm regardless of cycle count. This catches accelerated early failures the wear model would miss.



Figure 3: Three-zone alarm system based on health index percentage

## 4.2 MOSFET Thermal Fatigue Model

MOSFETs fail primarily through bond-wire fatigue from repeated thermal cycling. Modelled using the Norris-Landzberg power cycling equation — the industry standard for semiconductor thermal fatigue.

- Each power cycle is characterised by its junction temperature swing (delta Tj) and peak junction temp
- Damage scales as delta Tj raised to approximately the power of 5.4 — large swings are disproportionately harmful
- Miner's rule accumulates damage: each cycle consumes 1/Nf of total life budget
- End-of-life when accumulated damage fraction reaches 1.0

## 4.3 LCD Backlight Model

Backlight life follows the L70 lumen depreciation standard — end-of-life when output falls to 70% of initial value. Runtime hours are accumulated and adjusted by a temperature acceleration factor.

## 4.4 Mechanical Switch Model

Switches are monitored by counting actuations and tracking contact bounce duration. Rising bounce time indicates mechanical wear independently of cycle count and acts as an early warning override.

## 5. Communication Layer

Data flows from the inverter to the cloud through two protocol layers: MODBUS for the local hardware link and MQTT for the internet transport.

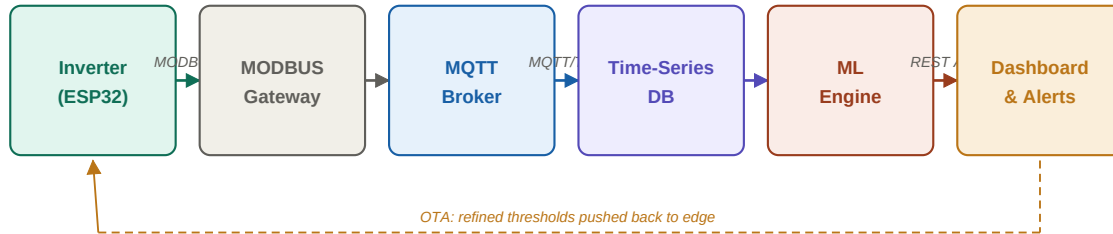


Figure 4: End-to-end data pipeline from inverter to cloud dashboard

### 5.1 MODBUS Protocol (Inverter to Gateway)

- MODBUS RTU over RS-485 for wired connections, or MODBUS TCP over WiFi for wireless
- Gateway polls all health registers on a configurable 5-to-60-second interval
- All component health data exposed as holding registers at fixed addresses

#### MODBUS Register Map:

| Register | Name         | Type   | Unit   | Description              |
|----------|--------------|--------|--------|--------------------------|
| 40001-2  | Relay_ops    | UINT32 | count  | Lifetime operation count |
| 40003    | Relay_HI     | UINT16 | x1000  | Health index 0-1000      |
| 40005    | Relay_alarm  | UINT8  | 0/1/2  | 0=OK, 1=Warn, 2=Critical |
| 40007    | Coil_temp    | INT16  | 0.1 C  | Coil temperature         |
| 40009    | MOSFET_Tj    | INT16  | 0.1 C  | Junction temp estimate   |
| 40011    | MOSFET_life  | UINT16 | x10 %  | Life consumed percentage |
| 40013    | LCD_hours    | UINT32 | hours  | Backlight runtime total  |
| 40015    | RUL_days     | UINT16 | days   | Cloud-estimated RUL      |
| 40017    | Fault_bitmap | UINT16 | bitmap | One bit per component    |

Table 2: MODBUS holding register map for component health data

### 5.2 MQTT Protocol (Gateway to Cloud)

- Topic structure: inv/{device\_id}/component/{id}/rul
- QoS level 1 ensures delivery even on unreliable networks — no readings silently lost
- TLS 1.3 encryption with device certificates authenticates each gateway
- Gateway buffers readings in local SQLite when connectivity drops; replays on reconnect

## 6. Cloud Machine Learning Layer

The cloud layer provides intelligence that is impossible on a constrained edge device. It learns from historical data across the entire fleet and continuously improves RUL accuracy.

### 6.1 LSTM Autoencoder — Anomaly Detection

Trained on health trajectories from known-healthy components, this model learns the normal shape of degradation curves. When a component behaves abnormally, reconstruction error spikes typically 10 to 20 data points before any rule-based threshold fires — the AI early-warning advantage.

- Training data: synthetic healthy degradation curves plus historical field data
- Anomaly threshold: mean reconstruction error plus 3 standard deviations on training set
- Output: anomaly flag and severity score overlaid on the dashboard timeline

### 6.2 Weibull Survival Analysis — Probabilistic RUL

Rather than a single point estimate, the Weibull model gives a probability distribution over remaining life: '85% confidence this relay survives 8,000 more operations.' Parameters are fitted from fleet-wide field failure data and updated continuously.

- Shape parameter beta: characterises whether failure rate is increasing or decreasing
- Scale parameter eta: the characteristic life at which 63.2% of units have failed
- Output: survival probability curve with confidence bands on the dashboard

### 6.3 Adaptive OTA Feedback Loop

As the fleet accumulates real failure data, the cloud learns that field conditions differ from datasheet ratings and pushes corrected thresholds to all edge devices via OTA. Edge predictions improve over time without manual recalibration.

### 6.4 Work Order Automation

When RUL drops below a configurable horizon (e.g. 30 days), the cloud automatically creates a maintenance work order with the specific component, unit location, estimated time to failure, and recommended action — integrating with SAP PM, IBM Maximo, or webhooks.

## 7. Alarm Strategy

| Normal   | HI > 20%        | Green LED steady            | Dashboard green, no action          |
|----------|-----------------|-----------------------------|-------------------------------------|
| Warning  | 10% < HI <= 20% | Amber LED blinking (2 Hz)   | Dashboard amber, notify maintenance |
| Critical | HI <= 10%       | Red LED + continuous buzzer | Dashboard red, auto work order      |
| Override | Anomaly flag    | Amber or Red per severity   | LSTM alert + investigation flag     |

Table 3: Four-level alarm strategy across edge and cloud

**Critical Design Principle**

Critical design principle: the edge alarm must be self-contained. It reads from non-volatile memory on boot and drives LEDs and buzzer independently of network status. Cloud connectivity enhances the system but is never a dependency for basic safety alerts.



## 8. Hackathon Implementation Plan (1 Week)

With a one-week hackathon, ESP32 + Raspberry Pi hardware, and a team strong in frontend, ML/data science, and backend/cloud — the following plan maximises demo impact.

**Hardware role assignment:**

- **ESP32** — simulates the inverter. Potentiometer controls stress level. As stress increases, relay health degrades in real time. LEDs and buzzer provide physical alarm output. MODBUS TCP over WiFi exposes all registers.
- **Raspberry Pi** — acts as edge gateway. Polls ESP32 via MODBUS, runs the RUL algorithm, publishes to a local MQTT broker, and bridges to the cloud backend.

| Day      | Hardware (ESP32 + Pi)                                            | Backend / Cloud                          | ML + Frontend                                 |
|----------|------------------------------------------------------------------|------------------------------------------|-----------------------------------------------|
| Day 1    | ESP32 MODBUS registers + NTC LED/buzzer wiring                   | Pi + Mosquitto setup<br>FastAPI skeleton | Schema + mock data<br>React scaffold          |
| Day 2    | Degradation simulator<br>Potentiometer = stress                  | RUL algorithm on Pi<br>MQTT publish loop | LSTM training (synthetic)<br>Anomaly baseline |
| Day 3    | ESP32 <-> Pi integration<br>Verify MODBUS flow                   | Pi->MQTT->FastAPI->WebSocket             | Live dashboard gauges<br>RUL timeline chart   |
| Day 4    | Stress scenario scripts<br>Pre-record demo profiles              | Weibull API<br>Work-order trigger        | LSTM anomaly overlay<br>Weibull curve on UI   |
| Day 5    | Full integration test<br>Record backup video                     | Fix edge cases<br>Load test              | Dashboard polish<br>Slide deck                |
| Days 6-7 | DEMO: green -> amber -> red -> work order auto-created on screen |                                          |                                               |

Table 4: Day-by-day build schedule split by team skill area

### 8.1 Demo Scenario Script (3 minutes)

|        |                                                                                                                                                                        |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Step 1 | Start: all components at 100% health. Dashboard all green. ESP32 LED steady green.                                                                                     |
| Step 2 | Turn potentiometer to high stress. Health index begins dropping live on dashboard.                                                                                     |
| Step 3 | At 20%: amber LED blinks. Dashboard turns amber. LSTM anomaly flag appears on chart — before the rule-based threshold fires. Emphasise this gap to judges.             |
| Step 4 | At 10%: red LED + buzzer trigger. Dashboard turns red. Work-order card auto-populates: 'Replace Relay K1 — estimated failure in 2.1 hours — Unit INV-001'.             |
| Step 5 | Business case: unplanned downtime costs X per hour. This system predicts failure 48 hours early, enabling a planned replacement during a scheduled maintenance window. |

**Hackathon Tip**

Always record a backup video of the full demo on Day 5. If WiFi drops or hardware fails on demo day, play the video without breaking stride. Judges care about the concept and build quality — a backup demonstrates professionalism, not weakness.

## 9. Full Production Roadmap

### Phase 1 — Edge instrumentation — 3-4 months

- Instrument inverter firmware with cycle counters and stress-weighted wear models
- Add LED and buzzer alarm outputs driven by health index thresholds
- Implement non-volatile health register storage (EEPROM or Flash)
- Validate degradation models against accelerated life test data per component
- Expose all health data over MODBUS at the agreed register map

### Phase 2 — Connectivity — 2-3 months

- Integrate MODBUS polling dongle or gateway firmware on Raspberry Pi
- Stand up MQTT broker and time-series database (InfluxDB or TimescaleDB)
- Build a minimal live dashboard showing RUL percentage per unit
- Validate data pipeline end-to-end from inverter sensor to cloud display

### Phase 3 — Cloud ML — 4-6 months

- Collect 6+ months of fleet telemetry to build sufficient training data
- Train LSTM autoencoder on normal vs degraded health trajectories
- Deploy Weibull survival models fitted to actual field failure data
- Enable OTA threshold updates from cloud back to edge devices

### Phase 4 — Integration and scale — 2 months

- Connect work-order trigger to CMMS (SAP PM, IBM Maximo, or webhooks)
- Build fleet-wide RUL heatmap showing all units and their health status
- Add confidence intervals to all RUL estimates for maintenance planning
- Establish model retraining pipeline as new field failure data arrives

## 10. What Makes This System Compelling

| Dimension           | This system                                                                  |
|---------------------|------------------------------------------------------------------------------|
| Hardware visibility | Physical LEDs and buzzer make failure prediction tangible and real           |
| AI advantage        | LSTM flags anomalies before rule-based alarms — demonstrably earlier warning |
| Protocol standard   | MODBUS + MQTT are industrial standards — integrates with existing equipment  |
| Business case       | Unplanned downtime cost vs planned replacement during maintenance window     |
| Scalability         | Fleet-wide learning: every new failure improves predictions for all units    |
| Edge autonomy       | Edge alarms work without network — cloud enhances but never gates safety     |

### The Winning Formula

The three things judges and customers remember: real hardware doing something visible, a clean live dashboard that non-technical people can read at a glance, and a clear business case expressed in money and time saved. Have all three locked by Day 5.